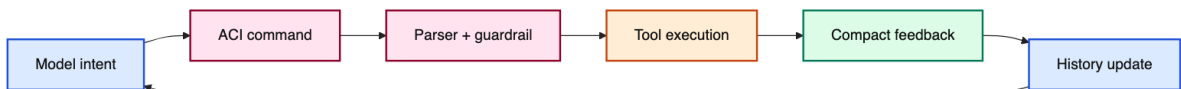


第 06 讲: SWE-agent: Agent-Computer Interface 为什么重要

Coding Agent Notes (June 2026)

1 核心图景

#05 已经把 coding agent harness 定义成 state、action、runtime、observation、trajectory、stop、validation 的闭环。SWE-agent 的贡献,是把其中最容易被低估的一层单独拎出来: **Agent-Computer Interface**, 也就是模型和计算机之间的交互界面。



ACI 不是“多给几个工具”。它定义模型能发出什么命令, 命令如何被解析, 哪些动作会被拦截, 动作在哪里执行, 执行结果怎样反馈, 旧观察如何压缩, 以及当前工作状态如何重新放回 prompt。换句话说, ACI 是把模型意图变成可靠环境操作的接口契约。

SWE-agent 的中心主张:

同一个语言模型, 如果通过更适合 LM 的接口与 repo 交互, 就能更稳定地搜索、阅读、编辑和验证代码。

这条主张比“给 agent 一个 shell”更细。raw shell 的能力很强, 但它默认用户是人类: 人可以视觉扫描长文件, 记住编辑器状态, 熟悉几十个命令选项, 知道空输出意味着什么, 也能从坏编辑里手动恢复。LM 并不天然具备这些交互习惯。SWE-agent 的 ACI 就是在修复这个 mismatch。

1.1 ACI 和 prompt engineering 的边界

prompt engineering 主要改变模型在 token 空间里的行为: 如何解释任务、如何计划、如何输出。ACI 则改变模型和外部环境之间的操作界面: 动作是否容易调用, 执行是否安全, 反馈是否可读, 状态是否显式。两者会互相影响, 但不是同一件事。

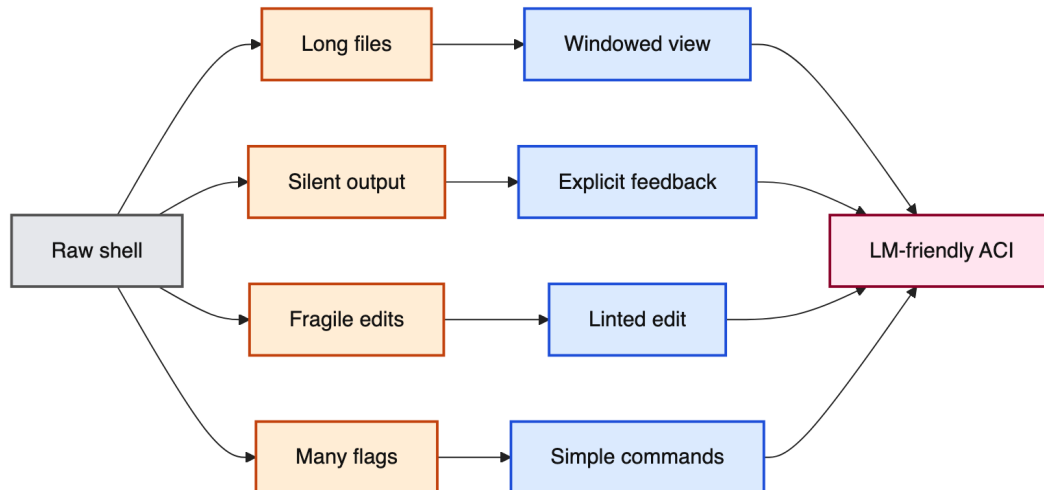
如果只改 prompt, 模型可能更认真地写“我会先搜索相关文件”, 但它仍然要自己决定用什么 grep 命令、如何避免输出过长、如何定位文件窗口、如何编辑多行。ACI 把这些高频操作变成稳定原语, 让 prompt 可以把注意力放在策略上, 而不是命令细节上。

层次	主要问题	典型改法
Prompt	模型怎样思考和表达	prompt / demos / format
ACI	模型怎样改变计算机状态	tools / docs / guardrails
Runtime	动作在哪里产生副作用	container / shell / timeout
Validation	什么算完成	tests / submit / patch

SWE-agent 的论文价值就在于它把 ACI 当成独立研究对象: 固定模型能力的大方向, 观察接口变化如何改变 agent 行为。这让我们能把“模型强”与“界面设计好”分开分析。

2 为什么 raw shell 不够

Linux shell 对人类工程师是强大界面，但对 LM agent 是高摩擦界面。它把太多低层操作暴露给模型：搜索文件、筛选结果、分页、定位行号、构造多行编辑、处理命令沉默、解释错误输出、记住当前文件状态。模型一旦在这些低层操作上出错，就会把推理预算浪费在和界面搏斗上。



Human UI 假设	LM 常见弱点	SWE-agent ACI 修复
能视觉扫完整文件	token window 有限	windowed file viewer
能理解沉默命令	空输出容易误读	显式 no-output message
能熟练写 shell flags	命令格式脆弱	简短工具签名和文档
能手动恢复坏编辑	错误会级联	lint guardrail 和 undo
能筛选大量搜索结果	容易过度探索	capped summary search

SWE-agent 论文用 shell-only baseline 做了一个很重要的对照：模型不是没有能力，而是界面让它把能力花在了错误层级。比如在 shell-only 环境中，模型可能用 cat 打出大文件，把上下文窗口冲满；用 sed 做多行编辑时犯引号或行号错误；命令没有输出时误以为失败；一个小语法错误导致后面多轮调试都偏离。

2.1 一个 shell-only 失败链

设想一个普通 issue：某个 parser 在边界输入上失败。shell-only agent 可能先用 grep 搜 parser 名称，得到几十个文件；它用 cat 打开一个长文件，结果关键函数淹没在几百行输出里；它尝试用 sed 替换几行代码，但行号因为前一次插入已经变化；测试失败后输出里既有原始 bug，也有它新增的 syntax error。此时模型看见的是混杂日志，很难判断下一步该定位 bug 还是修复自己造成的错误。

这条失败链的每一步都不是“模型不会编程”。它是界面没有帮助模型做正确的状态管理：搜索没有摘要，查看没有窗口，编辑没有 guardrail，反馈没有区分原始失败与新增失败。ACI 的设计目标，就是在这些关节点上切断错误级联。

Insight. ACI 的价值不是替模型解决 bug，而是减少“界面诱发错误”。当搜索、查看、编辑和反馈都更稳定时，模型才有机会把推理能力用在真正的软件问题上。

3 ACI 的三个原则

SWE-agent 论文把 ACI 设计总结成三个原则。它们看起来简单，但几乎贯穿所有工具实现。

动作要简单。

一个动作应该容易被模型理解和调用。很多 bash 命令本身很强，但选项多、语义密、组合空间大。ACI 命令需要把常见软件工程操作封装成少数清楚动作，例如 find_file、search_file、search_dir、open、goto、edit、submit。

动作要紧凑。

紧凑不是少做事，而是把一个工程上自然连续的动作合并成一次可靠交互。比如编辑文件时，人类会查看目标行、替换文本、检查结果、发现语法错误再撤销。SWE-agent 的 edit interface 把这些步骤压缩成“line-range edit + post-edit view + lint feedback”。这减少了模型需要手动维护的中间状态。

反馈要有信息但简洁。

反馈必须足够支持下一步行动，但不能把上下文窗口塞满。搜索结果要能指示哪些文件值得打开；编辑失败要指出新增 syntax error，并展示 proposed/original window；命令空输出要明确说明成功但无输出；长观察要截断并提示模型换一种更窄的命令。

ACI = action vocabulary + command docs + parser + guardrails + feedback templates + history processing.

这也是 ACI 和普通 tool list 的区别。tool list 只告诉模型“可以调用什么”；ACI 还规定调用失败怎么办、输出怎么变成 observation、历史怎么保留、状态怎么暴露给下一轮。

3.1 好命令应该像低噪声 API

一个 ACI 命令可以按 API 来评估。它应该有清晰名称、少量参数、明确前置条件、可预测输出、可解释错误和低副作用。比如 search_dir 的前置条件是给定搜索词和可选目录；输出是文件级摘要；如果结果太多，它要求缩小搜索，而不是返回巨大列表。相比之下，裸 grep 的输出量、排序、上下文和二进制文件行为都需要模型自己控制。

命令品质	好接口	坏接口
名称	直接表达意图	依赖复杂 flag 组合
参数	少且类型清楚	字符串拼接和 shell quoting 多
输出	短、稳定、可行动	长、随机、需要二次解析
失败	给出下一步提示	只返回模糊错误码
副作用	局部、可撤销、可审计	隐式改变大量状态

这个标准也解释了为什么 ACI 不是把所有能力都封装成万能工具。万能工具的参数和输出通常很复杂，模型仍然要在一个大界面里推理。SWE-agent 选择的是少数高频、低噪声、可组合的工程动作。

4 Search 和 navigation

repo repair 的第一步通常不是写 patch，而是定位。SWE-agent 为定位提供三个搜索命令。

命令	查询对象	输出形状
find_file	文件名	匹配路径列表
search_file	当前或指定文件里的字符串	匹配行号和行内容
search_dir	目录里的字符串	每个匹配文件及匹配次数

这些命令的重要性不只是“搜索更方便”。它们在塑造模型的探索策略。raw grep 可能返回大量上下文，模型会被迫在长日志中筛选；iterative search 一次显示一个匹配，模型可能反复 next，直到耗尽预算；summarized search 则先给一个全局但短的候选集合，让模型决定下一步打开哪个文件。

repo 里的 search_dir 会统计匹配文件和匹配次数。如果匹配文件超过阈值，它不会把所有结果都打印出来，而是要求模型 narrow search。search_file 对匹配行数也有类似约束。这种限制不是为了藏信息，而是为了防止模型被低价值结果淹没。

4.1 搜索结果是一种 policy hint

搜索工具的输出形状会影响 agent 后续动作。若输出是“所有匹配行加上下文”，模型容易沿着每个局部上下文逐个看；若输出是“文件名 + 匹配次数”，模型会先做文件级选择；若输出直接打开第一个结果，模型可能过早锚定。

因此，ACI search 不是普通检索，而是把代码库探索变成更可控的策略。它应该让模型先粗定位，再细定位：先找到可能相关文件，再进入文件窗口，最后用行号和局部上下文决定编辑范围。

4.2 从 issue 到文件的典型路线

一个 SWE-agent 轨迹常见的定位路线可以读成三段。

第一段是词汇锚定。模型从 issue 中抽取类名、函数名、异常名、错误消息、配置项等搜索词。此时 find_file 和 search_dir 帮它把自然语言问题投射到 repo 文件集合上。

第二段是文件级筛选。search summary 返回多个候选文件，模型根据路径、匹配次数、文件名语义选择少数文件打开。这个阶段最怕输出过多，因为过多候选会诱导模型逐个探索，而不是形成假设。

第三段是局部确认。打开文件后，模型用 search_file、goto 和窗口上下文确认具体函数或分支。只有当局部语义和 issue 对上时，才应该进入编辑。

这条路线的重点是“逐步缩小不确定性”。一个好 ACI 不应该让模型在第一步就面对整仓库，也不应该让模型在还没理解局部语义时直接写 patch。

5 Windowed file viewer

长文件是 LM agent 的典型难点。人类可以用 IDE 折叠、滚动、视觉记忆来读文件；模型看到的是 token 流。SWE-agent 的 file viewer 用窗口把长文件切成稳定局部上下文，并把当前文件状态显式化。

repo 的 windowed 工具定义了 open、goto、create、scroll_up、scroll_down。打开文件后，工具维护当前文件和窗口位置。论文中默认窗口最多显示 100 行，并配合行号、当前路径、上下文提示，让模型知道自己正在看哪里。

Viewer 机制	解决的问题	设计取舍
固定窗口	防止整文件冲垮上下文	需要滚动和跳转
行号	支持稳定编辑范围	行号会随编辑漂移
当前文件 state	减少 hidden editor state	依赖 state command
pre/post 提示	知道上下还有内容	不是完整语义视图

窗口大小也不是越大越好。太小，模型看不到足够上下文；太大，模型把注意力分散到无关代码。SWE-agent 的实验显示 window size 是真实影响性能的接口参数。这个结果说明：上下文管理不是模型外部的细枝末节，而是 agent 能力的一部分。

5.1 窗口让隐藏编辑器状态变成显式状态

人类使用 IDE 时，当前文件、光标位置、折叠状态、最近查看位置都在视觉界面里。模型没有这种视觉连续性。SWE-agent 的窗口工具把其中一部分状态写进环境 registry，并通过 state command 反馈给 prompt 模板，例如当前打开文件和工作目录。

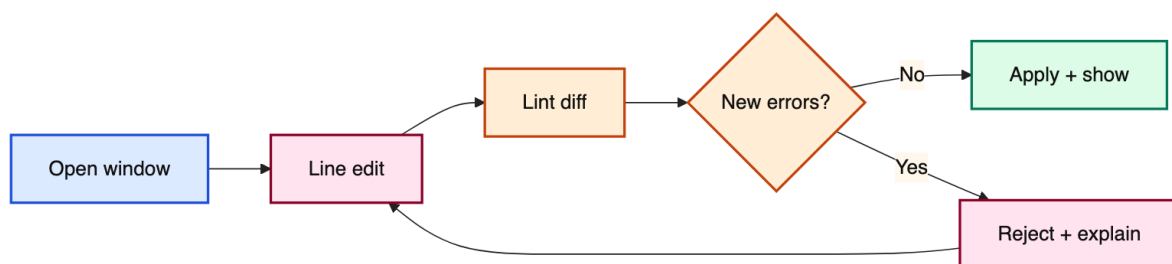
这件事很小，但很关键。没有显式 current file，模型可能以为自己在编辑 A 文件，实际工具仍指向 B 文件；没有 line window，模型可能基于过期行号编辑；没有 post-edit view，模型不知道文件实际变成什么。窗口化 viewer 的本质，是把人类 GUI 中隐含的上下文转成 LM 能读的文本状态。

5.2 窗口不是检索器

窗口化查看和 repo retrieval 不是同一层。retrieval 负责把候选上下文找出来；viewer 负责让 agent 在一个具体文件内部稳定操作。一个系统可以有很强的 retrieval，但如果打开文件后的行号、窗口位置和编辑反馈不稳定，最终 patch 仍然会失败。SWE-agent 的 ACI 强调的是后者：进入文件之后如何可靠工作。

6 Edit command 和 lint guardrail

编辑是 repo repair 中最脆弱的环节。模型可能找对 bug，也可能写出正确 patch 思路，但在把 patch 落到文件时失败：行号错、缩进错、括号多一个、多行替换范围错、重复插入、局部语法破坏。SWE-agent 把 edit 设计成 ACI 的核心组件。



edit 的接口是 line-range replacement：模型在当前打开文件上指定起止行和 replacement text。这个设计把“定位文件、定位行、替换文本、查看结果”合成一个明确动作。编辑成功后，file viewer 自动显示更新后的窗口，模型不需要额外运行 cat 或重新打开文件来确认。

更关键的是 lint guardrail。repo 的 linted edit 工具会在编辑前后运行 flake8，计算新增错误。如果编辑引入新的 syntax error，它会显示错误信息、显示如果应用后会是什么样、显示原始代码窗口，然后撤销这次修改，要求模型修正 edit command。

没有 guardrail	linter edit	对轨迹的影响
坏 edit 留在文件中	坏 edit 被撤销	防止环境状态污染
模型不知道实际差异	显示 proposed/original	更容易修正下一步
语法错误级联	新错误被局部截获	减少失败滚雪球
需要手动检查	成功后显示窗口	降低额外动作成本

Insight. linter edit 的本质是把“错误恢复”内置到界面里。它不是替模型写代码，而是在模型把环境搞坏之前截住可检测的低层错误。

6.1 为什么 lint guardrail 比事后测试更早

测试是最终验证的一部分，但 edit guardrail 应该更早发生。原因是测试失败的解释空间太大：可能是原 bug 没修，可能是新 patch 逻辑错，可能是新增 syntax error，可能是环境问题。lint 只检查一类低层错误，但它非常便宜，而且能把错误归因保持在编辑动作本身。

这类似数据库系统里的约束检查：它不能证明整个事务业务正确，但能阻止明显破坏结构的写入。对 agent 来说，早期 guardrail 的价值是减少后续状态的不确定性。被拒绝的坏 edit 不会污染文件系统，也不会进入后续测试反馈里制造混淆。

6.2 guardrail 也要给修复信息

简单拒绝动作并不够。如果工具只返回“edit failed”，模型仍然不知道哪里错。SWE-agent 的 linter edit 会展示新增 lint errors、如果应用会出现的代码窗口、原始代码窗口，并明确说明改动没有应用。这种反馈包含三种信息：错误原因、错误后果、环境是否被改变。

这三个信息缺一不可。错误原因告诉模型该修哪里；错误后果让模型看到自己写出的文本；环境状态说明防止模型以为坏 edit 已经生效。好的 guardrail 必须同时保护环境和推进下一步修正。

7 Feedback 和 history

SWE-agent 对反馈的处理很具体。它不是把命令输出原样追加到 history，而是通过模板、截断、空输出提示、错误提示和 history processor 让 observation 更可行动。

在 repo 的 TemplateConfig 中，可以看到多个反馈模板：普通 observation、truncated observation、no-output observation、bash syntax error、timeout error。比如命令超过时间限制时，反馈会明确提示该命令被取消，并解释常见原因是交互式或需要用户输入。命令没有输出时，也会返回“运行成功但没有输出”的含义，而不是让模型猜。

反馈场景	ACI 处理	避免的问题
空输出	显式说明无输出	模型误判为失败
长输出	截断并提示换窄命令	上下文被日志占满
语法错误	不执行并显示 shell check	错命令污染环境
超时	取消并解释可能原因	无限等待或交互命令
旧观察	压缩为 omitted 行数	历史过长

history processor 是另一个关键层。论文中使用过“保留最近观察，压缩更旧观察”的策略；repo 里的 LastNObservations 会把旧 observation 替换为类似 Old environment output: (n lines omitted) 的短提示。这样做保留了动作顺序和近期反馈，同时避免历史中的大段日志一直占用上下文。

7.1 压缩要保留因果链

history compression 的目标不是写漂亮摘要，而是保留“做了什么，得到什么，为什么下一步应该这样做”。如果旧输出完全删除，模型会重复探索；如果旧输出完整保留，模型会被噪声拖慢。SWE-agent 的策略是折中：旧观察压成短行，最近观察保持完整。

这对 repo repair 很重要。一个成功轨迹通常先定位，再打开文件，再编辑，再运行测试，再提交。压缩不能破坏这个阶段性因果链，否则模型后面可能忘记自己为什么改某个文件，也可能重复运行已经证明无用的命令。

7.2 反馈模板是在教模型如何恢复

SWE-agent 的反馈模板不只是错误消息。它们会告诉模型下一步怎样改变行为。例如 observation 被截断时，模板提示模型使用更窄的命令，或者用 head、tail、grep、重定向等方式减少输出。命令超时，模板提醒模型可能用了交互式命令或等待用户输入。

这种设计有一个很重要的教育效果：模型不是只收到失败信号，还收到恢复策略。对长期 agent 来说，恢复策略比单纯失败状态更有价值，因为它能让模型把下一步行动从“继续猜”变成“换一种更窄、更可控的操作”。

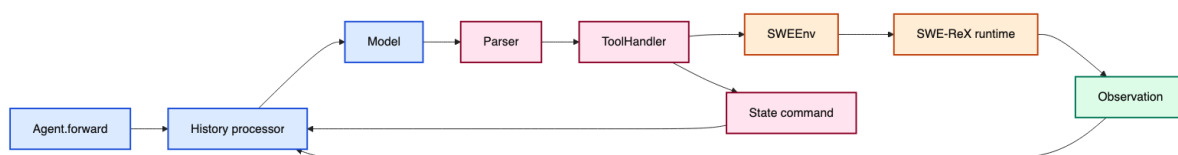
7.3 command blacklist 也是反馈设计

repo 的 ToolConfig 中有 blacklist，会拦截 vim、nano、less、tail -f、standalone shell、交互式 Python 等命令。这个设计不是限制模型能力，而是避免 agent 进入当前环境无法完成的交互模式。

被 block 的动作如果直接执行，agent 可能卡住、等待输入、输出不可控，或者让轨迹无法继续。拦截后返回明确错误，反而给模型一个可恢复的观察。这里再次体现 ACI 的原则：少给一些危险自由度，换取更稳定的闭环。

8 ACI 在 repo 架构里的位置

从 repo 架构看，ACI 不只是论文概念，而是贯穿 Agent、ToolHandler、SWEEnv 和 runtime 的实现层。



组件	作用	ACI 相关性
Agent.forward	调模型并执行动作	控制每一步 loop
HistoryProcessor	压缩 prompt history	管理上下文
Parser	从模型输出抽取 action	定义动作格式
ToolHandler	安装、解析、拦截工具	ACI 执行中枢
SWEEnv	管理执行环境	连接 runtime
state command	返回工作状态 JSON	暴露当前环境状态

tool bundle 让 ACI 可配置。一个 bundle 里有 executable、config、install script 和工具说明。config 中的 signature 和 docstring 会生成 command docs，帮助模型知道怎么调用。state command 则在每步之后返回当前工作目录、当前打开文件等状态，供 prompt 模板使用。

这说明 ACI 的工程实现至少包含四层：**文档层**让模型知道工具；**解析层**把模型文本变成可执行动作；**执行层**在环境中调用工具；**反馈层**把结果转换为 observation。任何一层不稳定，都会表现为 agent 能力下降。

8.1 parser 是 ACI 的语法边界

模型输出必须被解析成 action。SWE-agent 支持不同 parser 和 action format，包括 thought/action 风格、function calling、JSON 等。parser 的作用不只是抽字符串，它定义了哪些输出算有效动作、格式错时如何 requery、multiline command 如何结束、submit command 如何识别。

如果 parser 太宽松，模型的自然语言可能被误当命令；如果 parser 太严格，模型稍微偏离格式就无法行动。好的 parser 需要和工具文档、demonstration、format error feedback 配套，形成一个可学习的 action grammar。

8.2 state command 是 prompt 和环境的桥

工具 bundle 可以定义 state command，在每一步后返回 JSON 状态。这个机制很像把环境状态投影成 prompt 变量。当前工作目录、当前打开文件、窗口位置等信息不应依赖模型记忆，而应该由环境返回。

这比让模型“记住自己刚才打开了什么”可靠得多。长期任务里，模型可能忘记、总结错误、或被新 observation 干扰；state command 则把关键环境事实重新注入上下文。它是 ACI 中很容易被忽略但非常实用的一层。

9 Ablation 带来的设计课

SWE-agent 论文的 ablation 不是简单刷分，而是在回答“哪个界面设计真的影响 agent 行为”。几个结果尤其值得读。

No search 不只是少一个工具。

没有专门 search，模型仍可以用 grep、find、ls。但 raw shell search 会把太多格式和筛选工作交给模型。专门 search 的价值在于把 repo 定位过程变成短反馈、可选择、可继续缩小范围的交互。

Iterative search 可能比没有 search 更糟。

如果搜索结果一次只给一个，模型可能机械地 next、prev，试图看完所有匹配。这会消耗预算和上下文。这个现象说明，工具看似更像人类 UI，不一定更适合 LM。LM 需要的是能帮助它做决策的结果形状。

No edit 会把模型拖回脆弱 shell。

没有 line-range edit，模型只能用重定向、sed、临时文件等方式改代码。多行编辑会变得繁琐，错误不可见，后续观察也不稳定。编辑接口把一个高频、易错动作变成了稳定原语。

No linting 会让错误级联。

坏编辑如果直接落盘，后续测试失败可能同时包含真实 bug、语法错误和模型新引入的问题。linting 在编辑当下拦截低层错误，让 agent 不必在已经污染的环境里推理。

Full history 不是总是更好。

完整历史保留最多信息，但也保留最多噪声。长期 repo repair 需要的是近期观察和关键因果链，而不是所有日志。history processor 的存在说明，信息选择本身就是 ACI 设计。

9.1 分数背后的行为解释

SWE-agent 论文报告了 full SWE-bench 和 SWE-bench Lite 上的结果，并用 ablation 展示 ACI 组件带来的差异。读这些数字时，重点不是背具体百分比，而是理解数字背后的行为变化：模型是否更快定位；是否更少陷入搜索循环；是否更少产生不可恢复编辑；是否能在预算耗尽前提交；成功轨迹是否有稳定阶段结构。

这就是 ACI 研究和普通 leaderboard 的不同。leaderboard 只告诉我们结果，ACI 分析告诉我们为什么结果变了。一个接口改动如果让模型少犯低层错误，它可能比一次 prompt 微调更稳定；反之，一个看似强大的工具如果诱导模型过度探索，可能让性能下降。

9.2 成功快，失败慢

论文中还观察到一种很实用的现象：成功实例往往比较早提交，失败实例更容易拖到更高成本或更多步数。这不只是 benchmark 统计，也是 ACI 设计信号。一个好的界面应该让“有希望的路线”尽快收敛，也应该让“无希望的路线”暴露失败，而不是让 agent 长时间在噪声中徘徊。

这对 stop policy 和 observation 设计都有启发。如果 agent 连续多次 failed edit、连续扩大搜索范围、或反复运行无关测试，harness 应该提供更强的引导或终止信号。ACI 不能只服务成功路径，也要让失败路径更可诊断。

10 ACI failure map

读 SWE-agent 时，一个有用方法是把失败映射到接口层，而不是一律归因于模型。

失败表现	可能原因	ACI 修复方向
找不到相关文件	搜索结果太长或太碎	summarized/capped search
反复打开无关位置	文件级定位不足	先粗搜再窗口化查看
编辑后语法坏掉	raw edit 无检查	linted edit + undo
命令沉默后乱猜	observation 不明确	no-output template
上下文被日志冲满	feedback 太原始	truncation + history processor
模型用交互命令卡住	shell 权限过宽	blocklist + timeout
忘记当前打开文件	hidden state 不透明	state command

这个 failure map 的意义是：如果一个 agent 没有解决 issue，下一步不一定是换更强模型。可能应该先改搜索输出、编辑 guardrail、history compression、state feedback 或命令 blocklist。ACI 让我们有了调试 agent 行为的具体抓手。

10.1 评估 ACI 时应该报告什么

如果我们要比较两个 ACI，只报告最终 resolved rate 不够。至少还应该报告几类中间行为。

- **定位效率**：从开始到首次打开相关文件用了多少步，搜索结果是否过多。
- **编辑稳定性**：edit 成功率、failed edit 恢复率、lint error 是否级联。
- **观察质量**：输出截断次数、空输出误解、超时和 blocklist 命中次数。
- **上下文压力**：history 长度、旧观察压缩比例、上下文溢出次数。
- **提交行为**：成功提交发生在第几步，失败是否预算耗尽。

这些指标能帮助我们判断接口到底改善了什么。如果 resolved rate 上升但编辑失败也上升，可能只是模型或测试分布变化；如果定位步数下降、failed edit 降低、成功提交更早，那么 ACI 改动才更可信。

Insight. ACI 的评价应该看 trajectory，不只看最终 patch。因为 ACI 改变的是 agent 如何行动，trajectory 才能暴露这种改变。

11 设计 ACI 的验收清单

把 SWE-agent 的经验迁移到自己的 agent 时，最容易犯的错误是直接复制工具名，而不是复制接口约束。一个工具叫 edit 并不自动等于好 ACI；真正重要的是这个工具有没有让模型更少维护隐式状态、更少处理低层格式、更容易从错误里恢复。

检查项	应该满足的条件	不满足时的症状
动作语义	一个命令对应一个清楚工程意图	模型把 shell 拼接当成主任务
参数边界	参数少、类型明确、失败可解释	quoting、路径、行号错误频繁
状态显式化	当前文件、目录、窗口、diff 可见	agent 基于旧状态继续工作
反馈可恢复	错误说明原因、后果、是否生效	下一步只能猜测或重复动作
副作用控制	高风险动作可拦截、可撤销、可审计	坏 edit 污染后续验证
历史策略	近期观察保真，旧观察保留因果线索	上下文被日志淹没或遗忘路线

这张清单也能解释为什么 ACI 是一个系统边界，而不是 prompt 附录。prompt 可以要求模型“小心编辑”，但如果工具不能显示 proposed diff、不能撤销坏 edit、不能告诉模型当前文件，模型仍然会在同样的状态陷阱里失败。ACI 的作用是把些要求做成环境事实，而不是只写成行为建议。

11.1 什么时候不要封装新工具

工具越多不一定越好。一个新工具只有在减少真实摩擦时才值得加入：它应该压缩高频、稳定、可审计的动作，而不是把少见操作包装成另一个复杂接口。如果某个动作很少发生、参数组合巨大、输出难以稳定摘要，直接保留 shell 或让模型写脚本反而更合理。

候选能力	适合封装成 ACI 工具	更适合保留自由操作
文件搜索	高频、输出可摘要、容易限流	临时复杂正则探索
窗口查看	状态稳定、行号可复用	一次性生成大报告
代码编辑	易错且需要 guardrail	大规模机械重构脚本
测试运行	可模板化、可截断、可归因	特殊交互式调试会话
环境操作	可恢复、权限明确	破坏性系统配置修改

因此，好的 ACI 不是“把 shell 消灭掉”，而是把最常见、最容易诱发模型错误的动作升级为低噪声协议，同时保留必要的通用逃生口。这样模型既不会被无限自由度淹没，也不会被过窄工具锁死。

Insight. ACI 的设计标准不是工具数量，而是每个工具是否降低了状态不确定性。一个工具如果让 agent 更难判断自己做了什么、环境变成什么、下一步该看哪里，它就不是好接口。

12 局限与开放问题

SWE-agent 的 ACI 思想很强，但它不是所有 coding agent 问题的通用答案。第一，ACI 主要优化的是模型和计算机之间的交互界面；如果任务本身缺少可运行环境、测试信号不稳定、issue 信息不足，单靠更好的 search/view/edit 工具仍然无法可靠判断补丁是否正确。

第二，ACI 的有效性依赖任务分布。仓库修复任务通常需要搜索、阅读、编辑和测试，因此 windowed viewer、linter 和 search summary 很自然；但对 UI 自动化、数据分析、长时间产品开发或多用户协作，agent 可能还需要 browser state、权限确认、持久化会话和任务切分。这些问题已经超出单一 ACI，可以进入平台层。

第三，接口设计本身也可能过拟合。一个工具如果太贴合某个 benchmark 的失败模式，可能在别的项目里变成额外约束。真正稳健的 ACI 不应只追求某个 resolved rate，而要能解释：它减少了哪类低层错误，保留了哪些必要自由度，又把哪些风险交给了验证层。

13 最终 takeaway

SWE-agent 的 ACI 可以概括成三句话。

- raw shell 是为人类 power user 设计的，不是为 LM agent 的 token、记忆和错误模式设计的。
- ACI 通过简单命令、紧凑动作、窗口化查看、linter、显式反馈和历史压缩，把低层界面错误变少。
- ACI 的价值在于缩短 intent 到 reliable observation 的路径，让模型把推理能力用于真实软件问题。

因此，读 SWE-agent 不能只记住一个 benchmark 分数。更重要的是理解它的接口思想：好的 coding agent 系统不是把模型直接扔进 shell，而是为模型设计一种软件工程 UX。这个 UX 越能利用模型擅长的语言和代码推理，越能补偿模型不擅长的隐式状态、长日志筛选和错误恢复，agent 的行为就越稳。

参考资料

- [1] [SWE-agent paper](#). Agent-Computer Interface concept, design principles, ablations, and SWE-bench results
- [2] [SWE-agent repository](#). tool bundles, history processors, parser/tool/runtime implementation
- [3] [SWE-agent documentation](#). ACI overview, architecture notes, and tool configuration documentation